

Fundamental CS for AI Era

Building human-centered digital
products in the AI era



I'm Zain Fathoni

Front-end developer based in Yogyakarta, Indonesia. Previously backend, manager, frontend, now fullstack.

Community builder · AI-assisted engineering practitioner · Mentor Spotlight at BUILD Camp.

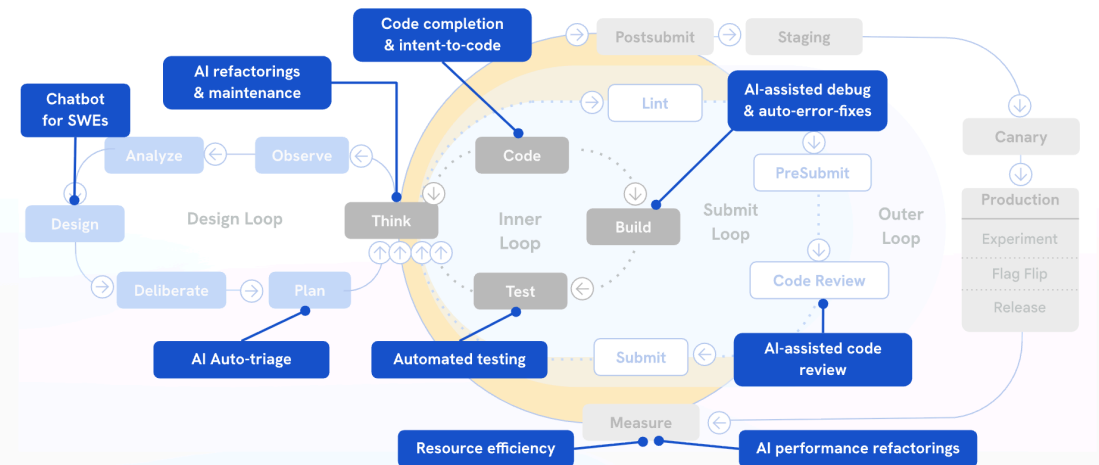
<https://zainf.dev>

Our work now demands AI fluency

Developers, designers, and product builders are all being asked to move faster.

Not someday. **This sprint.**

Where can AI improve developer experience?



This is the complete AI-assisted journey: design, code, review, and performance improvements.
Each opportunity where AI lifts developer productivity is fully mapped out.

Understanding is more important than ever

AI can generate code faster than we can review it.

The bottleneck moves from **typing** to **understanding what changed, why it works, and what it might break.**

Geoffrey Litt

Understanding matters not just to verify, but to participate.

From "Understanding is the new bottleneck"

But learning has never been easier

AI can explain code, generate examples, create quizzes, and build tiny tools to help us understand.

The opportunity: use AI as a **learning amplifier**, not just a code printer.

AI
×
CS

The win is not “skip fundamentals.” The win is “learn fundamentals faster, with feedback.”

P5 · FALSE HOPE

Does vibe coding even work, though?

It works until you hit the part where the codebase has history, constraints, edge cases, and users.

That is where lazy delegation becomes expensive.

4 July 2026 | Fundamental CS for AI Era

Prompt harder



Get more code



Understand less



Debug longer

What if fundamentals are the AI multiplier?

Fundamentals give you the words, mental models, and constraints to steer AI:

- data structures
- algorithms
- systems thinking
- debugging
- trade-offs

**AI TOOLS DON'T REPLACE
EXPERTISE - THEY AMPLIFY
IT. THE MORE SKILL YOU
BRING AS AN ENGINEER,
THE MORE POWERFUL THE
RESULTS YOU'LL GET.**

Read the code, but not all of it

The skill is not reading every line.

The skill is knowing **which code is worth reading** before you ask AI to change it.

Gergely Orosz

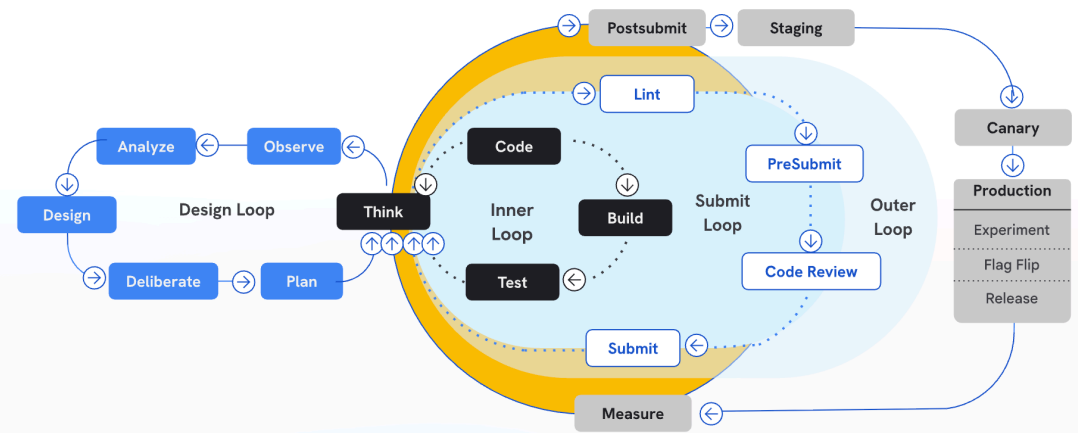
We read existing code much more often than we write new code.

[Related post on AI-assisted coding and reading code](#)

Here are the first things to do to get started...

1. **Research** the code before prompting.
2. **Plan** the exact behavior change.
3. **Implement** with AI in small loops.
4. **Verify** with tests, types, or screenshots.
5. **Explain** what changed in your own words.

Where can AI improve developer experience?



This is the baseline development loop without AI interventions.
It's useful as a reference to compare where AI automation and intelligence can be added to accelerate velocity.

Reaping early benefits

When you practice fundamentals with AI, you get compounding returns quickly:

- better prompts
- faster reviews
- fewer hallucinated fixes
- calmer debugging
- clearer product decisions

Instead of:

"AI wrote this; I hope it works."

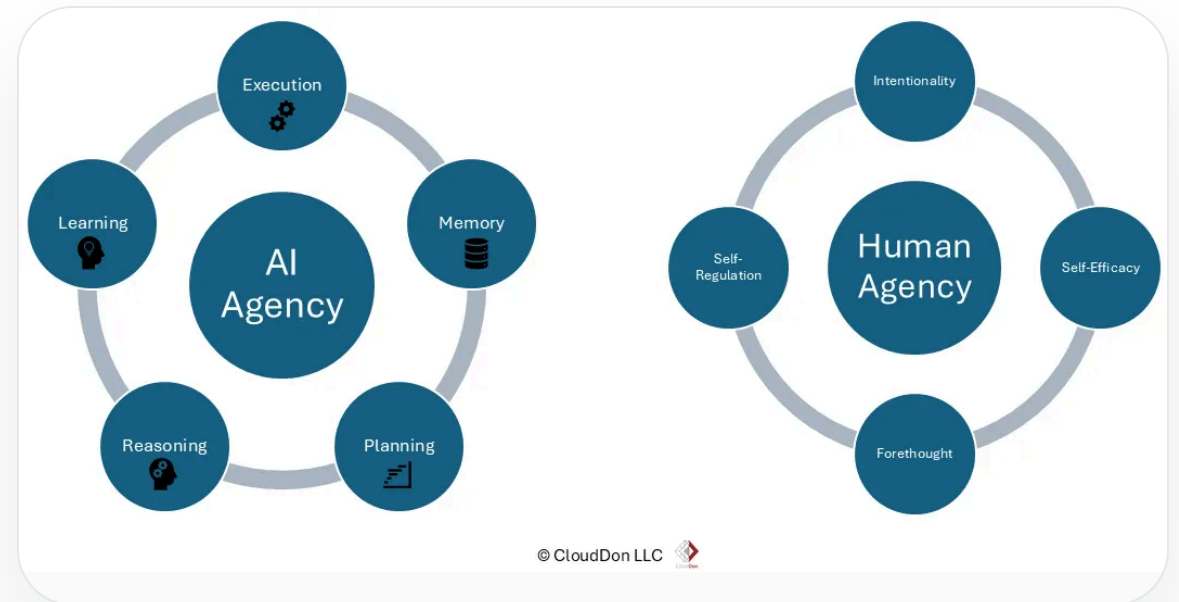
You can say:

"Here is the invariant, here is the trade-off, here is how we verified it."

Increasing the surface of agency

In the AI era, human-centered builders are not the people who type the most code.

They are the people who can frame problems clearly enough for humans and machines to solve together.



References

- Dan Roam — [The Pop-up Pitch](#)
- Geoffrey Litt — [Understanding is the new bottleneck](#)
- Gergely Orosz — reading code in the AI era
- Dex Horthy — research, plan, implement; avoid lazy AI loops
- Zain Fathoni — [A to Z #3](#)

**Fundamentals are not the past.
They are how we steer the future.**

Q & A

<<https://zainf.dev/fundamental-computer-science-ai-era>>

<<https://zainf.dev/a-z-3>>